

Exercises 4:

Part 1:

```
package edu.najah.cap.oop;

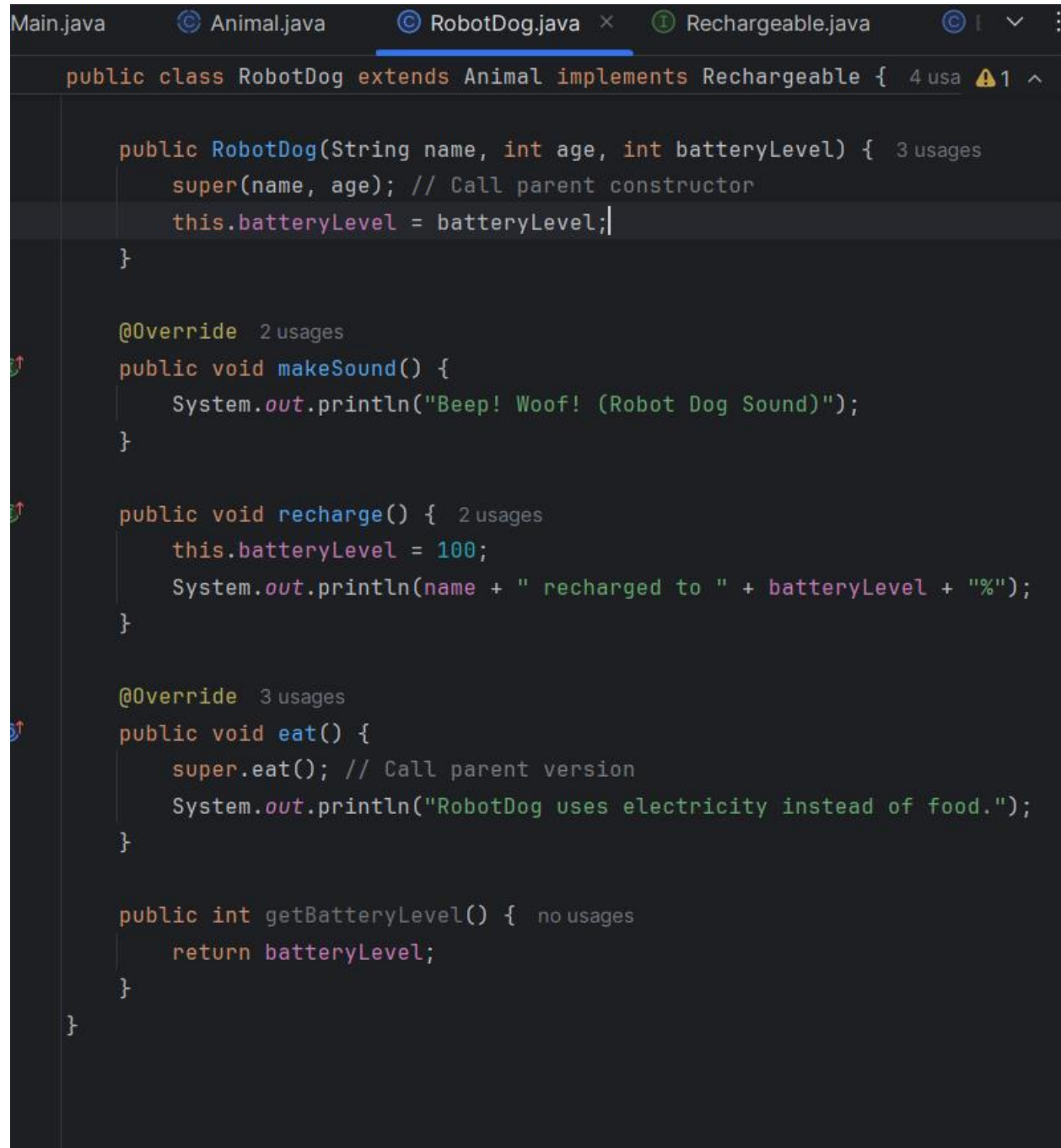
public abstract class Animal { no usages
    public String name;
    public int age; 1 usage

    public Animal(String name, int age) { no usages
        this.name = name;
        this.age = age;
        System.out.println(" Animal constructor called for " + name);
    }

    public abstract void makeSound(); no usages

    public void eat() { no usages
        System.out.println(name + " is eating...");
    }
}
```

Part 2 and part 3 :



```
public class RobotDog extends Animal implements Rechargeable { 4 usa ⚠ 1 ^

    public RobotDog(String name, int age, int batteryLevel) { 3 usages
        super(name, age); // Call parent constructor
        this.batteryLevel = batteryLevel;
    }

    @Override 2 usages
    public void makeSound() {
        System.out.println("Beep! Woof! (Robot Dog Sound)");
    }

    public void recharge() { 2 usages
        this.batteryLevel = 100;
        System.out.println(name + " recharged to " + batteryLevel + "%");
    }

    @Override 3 usages
    public void eat() {
        super.eat(); // Call parent version
        System.out.println("RobotDog uses electricity instead of food.");
    }

    public int getBatteryLevel() { no usages
        return batteryLevel;
    }
}
```

```
package edu.najah.cap.oop;  
  
public interface Rechargeable { 2 usages 1 implementation  
    void recharge(); 2 usages 1 implementation  
}
```

Part 4:

1.

الكلاس RobotDog عمل دالة makeSound(String sound) مختلفة عن الدالة الموجودة في الكلاس الأب makeSound() وهاد اسمه Overloading مش Overriding , ف بصير معناه انو ما تم استبدال الداله الاصليه من الكلاس الاب ف لما استخدم كائن من نوع RobotDog بمتغير نوعه Animal رح يتم استدعاء داله الاب مش داله الابن وكمان لما ما اعمل @Override بصير الخطا مش مبين بالكتابه .

2.

لأن ال overloading ينشئ دالة جديدة مختلفة عن دالة الأب وما بغير سلوك الاب اما ال overriding بعيد تعريف نفس الداله ب الكلاس الابن بنفس الاسم ونفس عدد وانواع البراميترز ف بسمح ب ال Polymorphism .

3.

بستخدم ال @Override وبكتب الداله ب الكلاس الابن

```
g.java x Rechargeable.java BadDesign.java FixedDesign.java x
C:\Users\Rahhal\IdeaProjects\Exercises4\src\edu\najah\cap\oop\RobotDog.java

class FixedAnimal { 1 usage 1 inheritor
    String name; no usages
    void makeSound() { no usages 1 override
        System.out.println("generic animal sound");
    }
}

class FixedRobotDog extends FixedAnimal { no usages
    int batteryLevel; no usages

    @Override no usages
    void makeSound() { |
        System.out.println("Beep! Woof! (Fixed RobotDog)");
    }
}
```

Part 5:

1.

`RobotDog dog = new Animal();` ما بتشتغل لانه ال `Animal` هو كلاس `abstract` ف ما بقدر اعمل كائن مباشرة منه .

`Animal dog2 = new RobotDog();` اشتغلت لانو `RobotDog` بورث من ال `Animal` ف بقدر اسند كائن الابن للمتغير من ننوع الاب .

```
Animal dog2 = new RobotDog( name: "Rex", age: 3, batteryLevel: 50);
dog2.makeSound();
dog2.eat();
```

الواجهة Rechargeable لانو الكائن يمكن انو يعامل كنوع الواجهة يلي بنفذها .
(implements) RobotDog لما ال RobotDog ينفذ

```
Rechargeable dog3 = new RobotDog( name: "Robo", age: 1, batteryLevel: 30);
dog3.recharge();
```

2.

RobotDog dog : يستطيع الوصول إلى كل الدوال الموجودة في RobotDog وفي Animal لأنها ترثها.

Animal dog2 : يستطيع الوصول فقط إلى الدوال الموجودة في Animal بس عند التنفيذ يختار نسخه الداله من RobotDog .

Rechargeable dog3: بقدر يوصل فقط إلى دوال الواجهة Rechargeable .

3.

رغم انو نوع المتغير هو Animal الا انو الكائن الحقيقي هو من نوع RobotDog عشان هيك يتم تنفيذ الداله الموجوده في RobotDog وليس في Animal وهاد الاشئ بسبب مبدا Polymorphism الذي يحدد اي داله بتننفذ حسب نوع الكائن مش ننوع المتغير .

4.

الـ Polymorphism بخلي جافا تختار اي نسخه من الداله بتننفذ بوقت التشغيل حسب نوع الكائن الفعلي ومش نوع النتنغير المصرح فيه هيك بسمح بكتابه كود عام بتعامل مع انواع متعدده من الكائنات بطريقه مرنه وقابله للتوسع .

```
Part : type assignment experiments
Animal constructor called for Rex
Beep! Woof! (Robot Dog Sound)
Rex is eating...
RobotDog uses electricity instead of food.
Animal constructor called for Robo
Robo recharged to 100%

Process finished with exit code 0
```

Step 6 :

1.

نسخة eat() الموجودة في RobotDog يتم تنفيذها لأنها قامت بعمل override لدالة الأب واستبدالتها بسلوك جديد واستخدمت super.eat() لاستدعاء الأصل .

2.

لأن نوع الكائن الحقيقي وقت التشغيل هو RobotDog وجافا بتطبق مبدأ Polymorphism الي يختار الدالة من الكلاس الفعلي للكائن مش من نوع المتغير .

3.

رح يبحث المترجم عن الداله في الكلاس الاب Animal وكم ان اذا كانت makeSound() موجودة فيه كـ (abstract) رح يظهر خطأ ترجمة لأن RobotDog لم يوفر تنفيذ لها .

```
package edu.najah.cap.oop;

public class Main {
    public static void main(String[] args) {
        System.out.println(" Part: creation and polymorphism ");

        RobotDog a = new RobotDog( name: "Bolt", age: 2, batteryLevel: 90);
        a.eat();
        a.makeSound();
    }
}
```

```
"C:\Program Files\Java\jdk-25\bin\java.exe" "-javaagent:C:\Program Files\JetBr
Part: creation and polymorphism
Animal constructor called for Bolt
Bolt is eating...
RobotDog uses electricity instead of food.
Beep! Woof! (Robot Dog Sound)
```

Reflecting :

1.

مبدأ ال Abstraction يساعد على فرض بنيه محدده للكلاسات وبجبرني ع تنفيذ الدول الاساسيه بس وبخليني اضيف انواع جديده بدون ما اغير الكود القديم يعني بمجرد ما ضفت نوع جديد زي ال Cat ما بحتاج اعيد كتابه نفس الكود من الصفر بس بنشئ كلاس جديد وبعمله extend من Animal .

2.

لو الكلاس Animal مش Abstract بصير بقدر اعمل منه كائن وهيك غلط لانو ال Animal هو عام ما بقدر اعمل كائن من اشي عام (الكلاس العام لازم يكون Abstract عشان امنع انشاء كائنات منه) .

3.

لانه بخلي الكود 1. مرن اكثر – بقدر اغير او ابدل الكلاسات بسهولة

2. Extensible بقدر اضيف انواع جديده بدون تعديل الموجود .

3. Reusable قابل لاعاده الاستخدام .

4. Testable بقدر اعمل اختبار لاجزاء النظام كل جزء لحال .

